



(12) **United States Patent
Marsh**

(10) **Patent No.: US 12,400,226 B2**
(45) **Date of Patent: *Aug. 26, 2025**

(54) **HIERARCHY-BASED BLOCKCHAIN**

(71) Applicant: **CITIBANK, N.A.**, New York, NY (US)

(72) Inventor: **Ryan Marsh**, Billerica, (GB)

(73) Assignee: **CITIBANK, N.A.**, New York, NY (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/662,096**

(22) Filed: **May 13, 2024**

(65) **Prior Publication Data**

US 2024/0296447 A1 Sep. 5, 2024

Related U.S. Application Data

(63) Continuation of application No. 18/136,595, filed on Apr. 19, 2023, now Pat. No. 11,983,711, which is a continuation of application No. 17/505,579, filed on Oct. 19, 2021, now Pat. No. 11,663,593, which is a continuation of application No. 17/096,620, filed on Nov. 12, 2020, now Pat. No. 11,676,144.

(51) **Int. Cl.**

G06Q 20/40 (2012.01)

H04L 9/06 (2006.01)

H04L 9/00 (2022.01)

(52) **U.S. Cl.**

CPC **G06Q 20/40** (2013.01); **H04L 9/0618** (2013.01); **H04L 9/0643** (2013.01); **H04L 9/50** (2022.05); **H04L 2209/56** (2013.01)

(58) **Field of Classification Search**

CPC **G06Q 20/40**; **H04L 9/0618**; **H04L 9/0643**
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

10,121,025 B1 * 11/2018 Rice **G06F 21/10**
11,488,161 B2 11/2022 Soundararajan et al.
2017/0243177 A1 * 8/2017 Johnsrud **G06Q 20/10**
2018/0253702 A1 9/2018 Dowding
2019/0251590 A1 8/2019 Bodington

(Continued)

FOREIGN PATENT DOCUMENTS

GB 2577751 A * 4/2020 **H04L 67/1097**
WO WO-2019/164926 A1 8/2019

Primary Examiner — Christopher Bridges

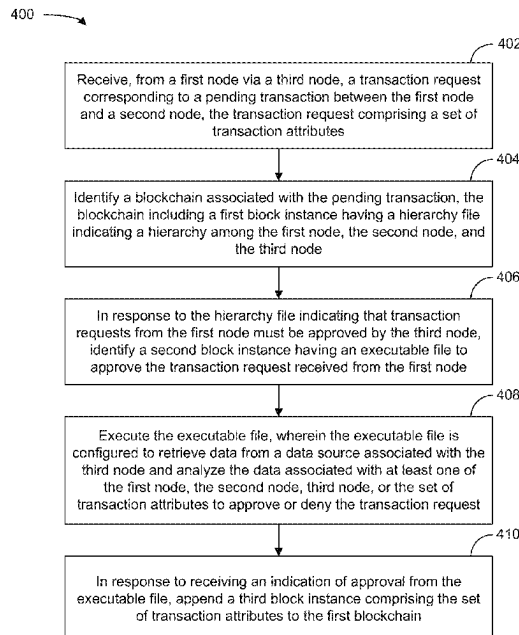
(74) *Attorney, Agent, or Firm* — Foley & Lardner LLP

(57)

ABSTRACT

A computer-implemented method comprising receiving a transaction request from a first node within the plurality of nodes, the transaction request corresponding to a pending transaction between the first node and a second node; identifying a distributed ledger associated with the pending transaction, the distributed ledger including a first record having a hierarchy file indicating a hierarchy among the first node, the second node, and the third node; identifying a second record having an executable file to approve the transaction request received from the first node; executing the executable file, wherein the executable file is configured to retrieve data from a data source associated with the third node and analyze data to approve or deny the transaction request; and appending a third record comprising the set of transaction attributes to the distributed ledger.

20 Claims, 5 Drawing Sheets



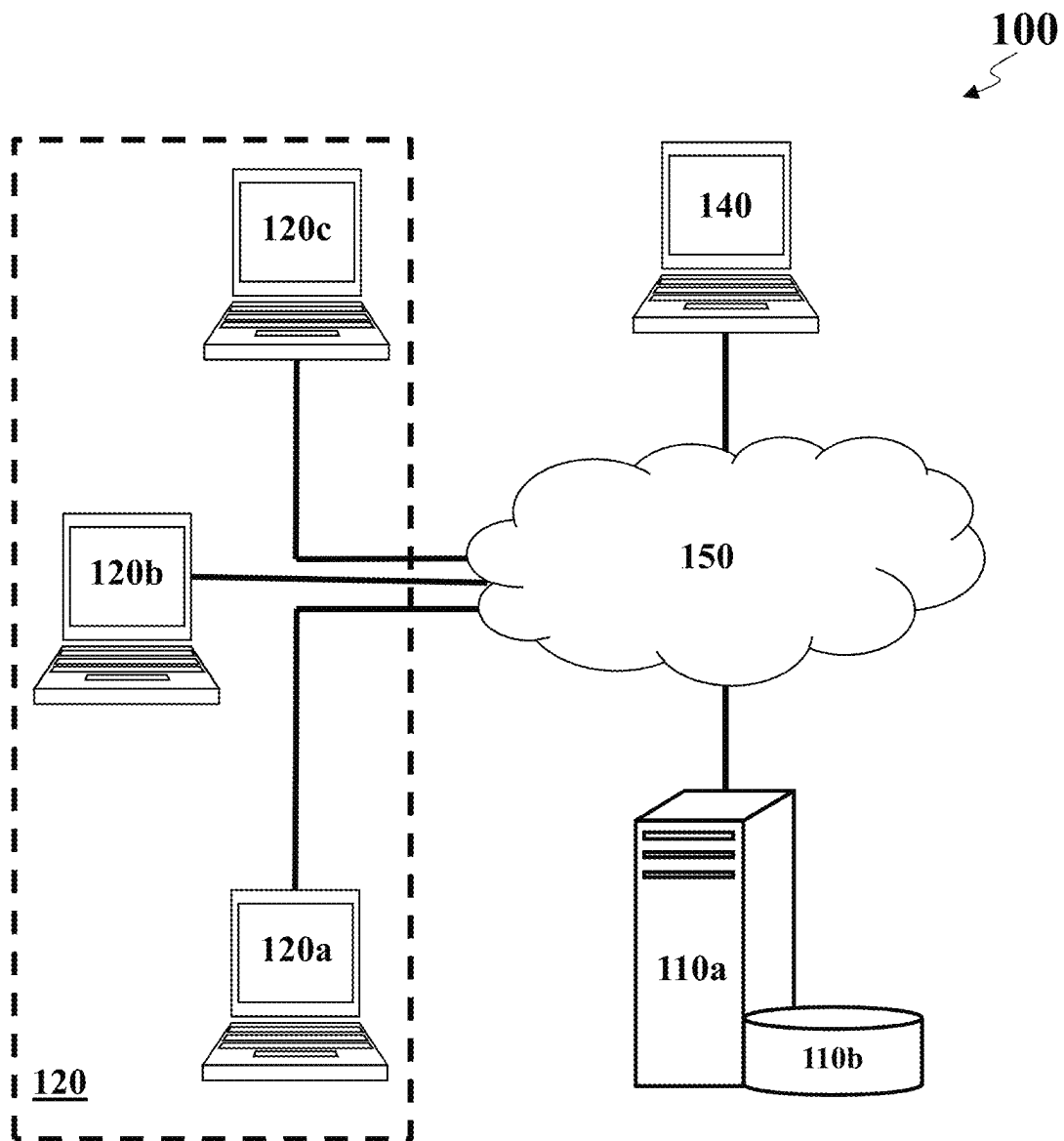
(56)

References Cited

U.S. PATENT DOCUMENTS

2019/0295050	A1 *	9/2019	Chalkias	G06Q 20/3829
2020/0051067	A1	2/2020	Overholser et al.	
2020/0119926	A1	4/2020	Buki	
2020/0410491	A1 *	12/2020	Ronnow	H04L 63/1441

* cited by examiner

**FIG. 1**

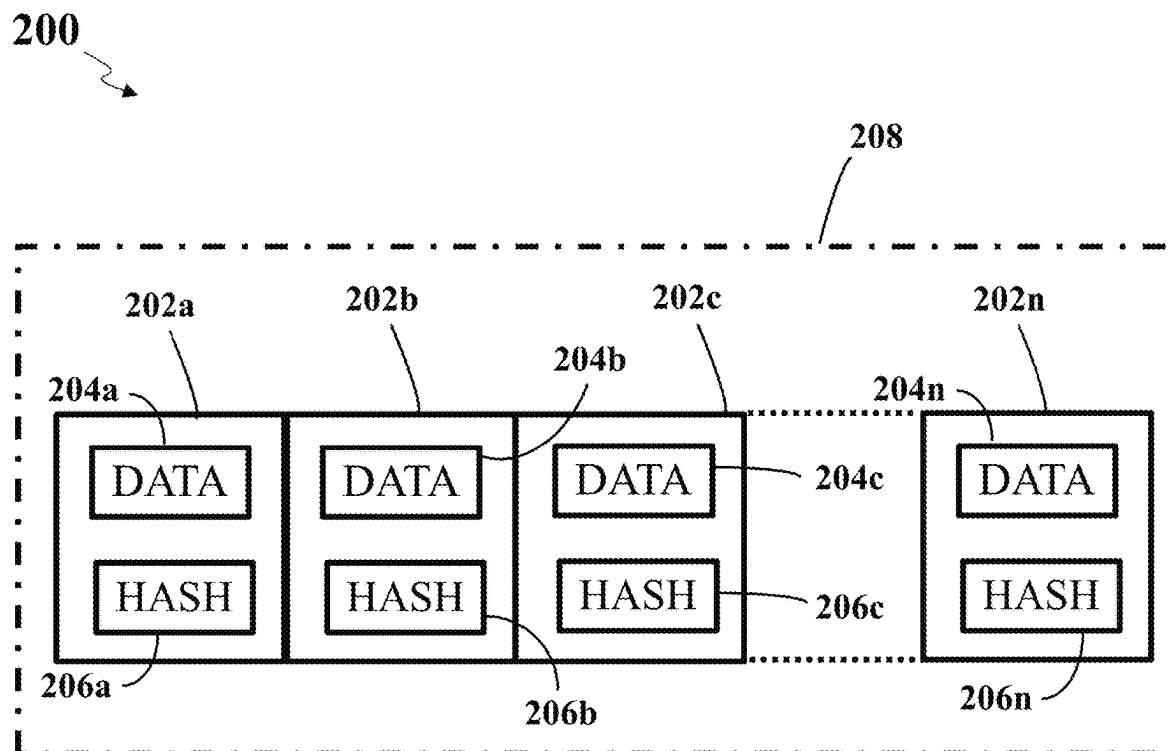


FIG. 2

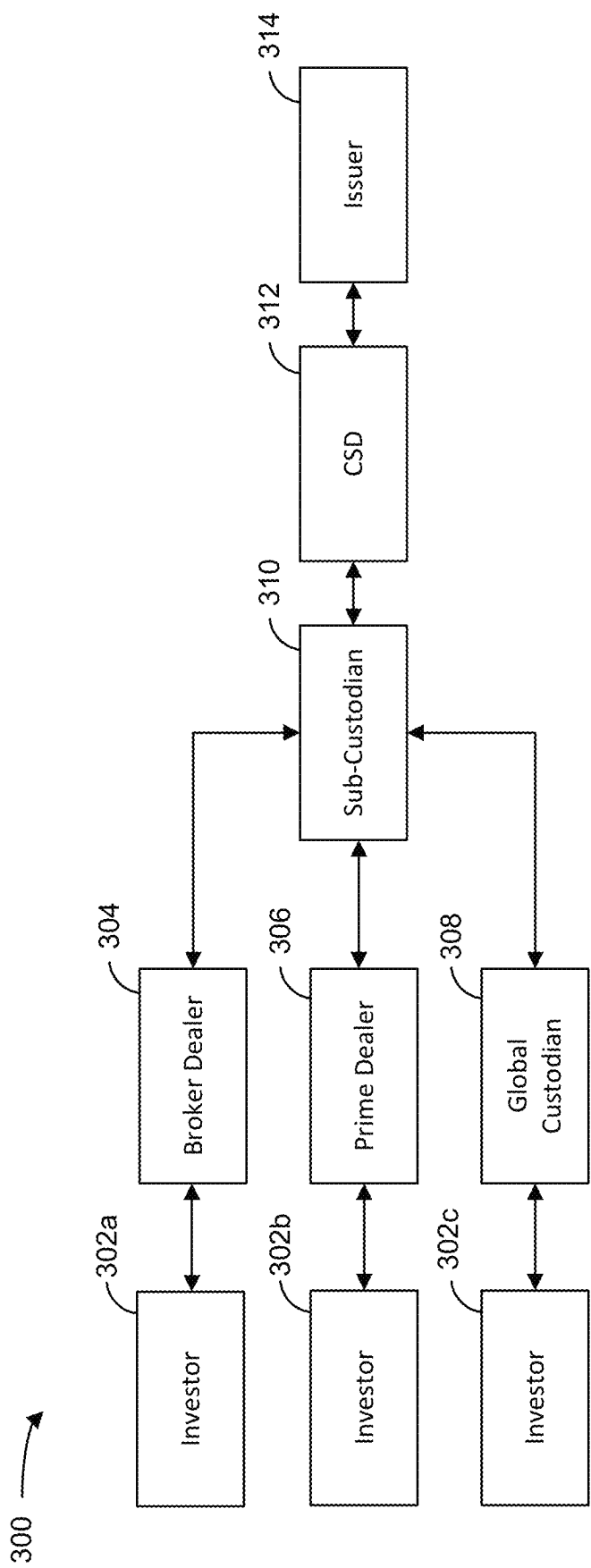


FIG. 3

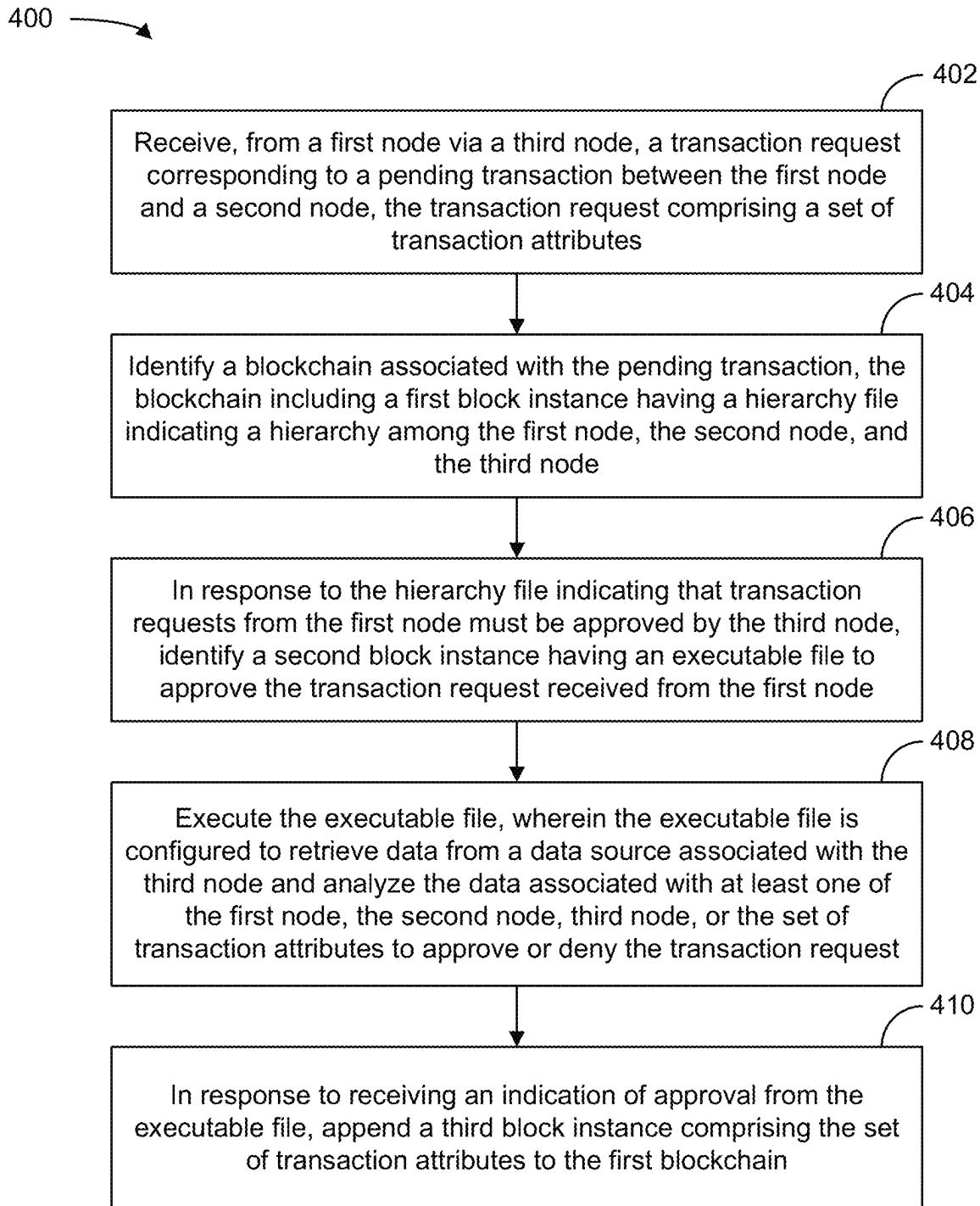
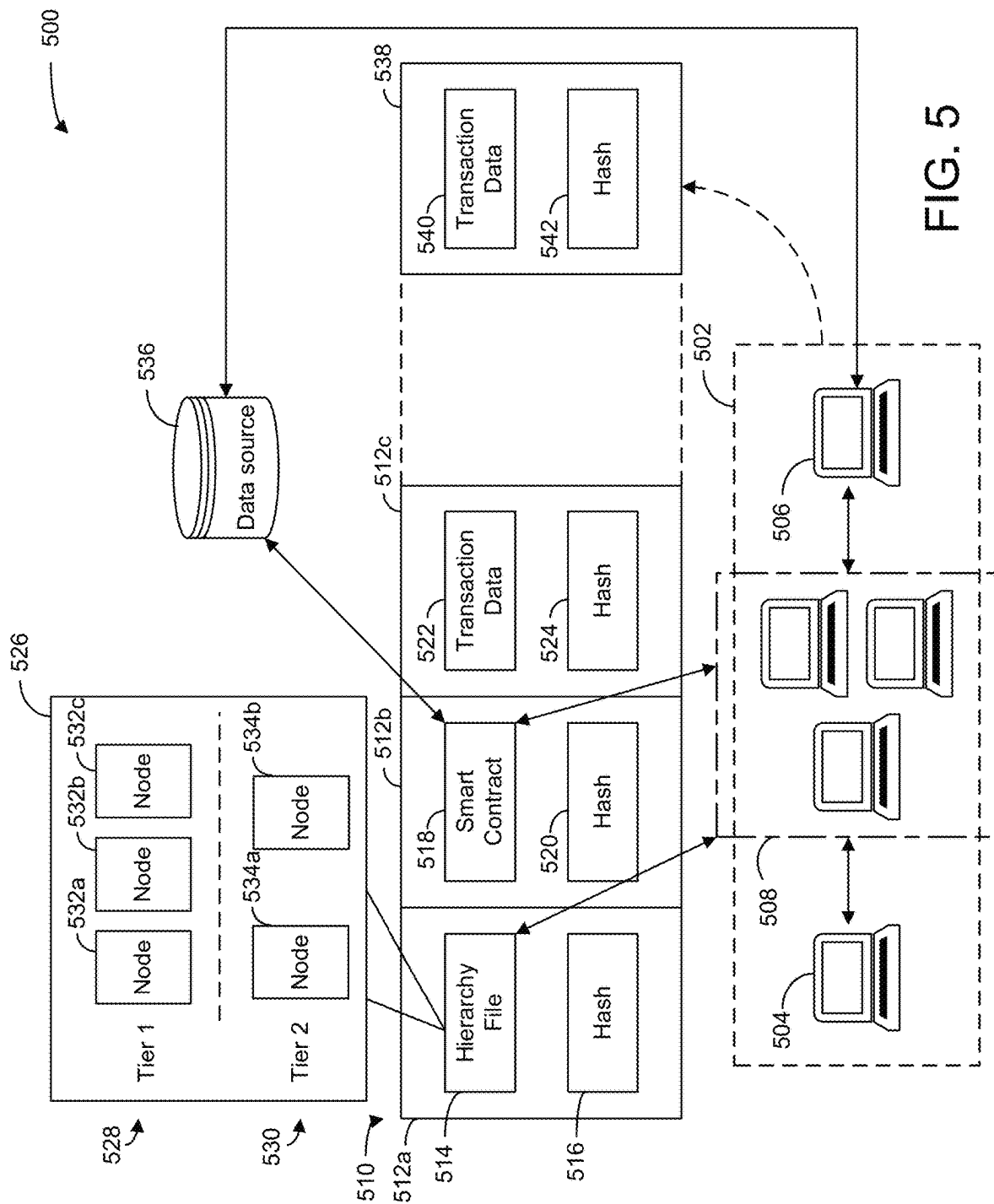


FIG. 4



1

HIERARCHY-BASED BLOCKCHAIN**CROSS-REFERENCE TO RELATED APPLICATIONS**

This application claims the benefit of priority as a continuation to U.S. patent application Ser. No. 18/136,595, filed Apr. 19, 2023, which claims the benefit of priority as a continuation to U.S. patent application Ser. No. 17/505,579, filed Oct. 19, 2021, now U.S. Pat. No. 11,663,593, which claims the benefit of priority as a continuation to U.S. patent application Ser. No. 17/096,620, filed Nov. 12, 2020, now U.S. Pat. No. 11,676,144, each of which is hereby incorporated by reference in its entirety.

TECHNICAL FIELD

This application relates generally to implementing a hierarchical data structure using blockchain-based technology.

BACKGROUND

As the processing power of computers allows for greater computer functionality and the Internet technology era allows for interconnectivity between computing systems, many institutions are shifting towards blockchain-based technology to store and maintain the integrity of transaction data. While blockchain-based storage methods may require more computational resources than conventional methods (e.g., a central database), the immutability of the data within the blockchain enables operators to have peace of mind that the data in the blockchain is accurate and will not be changed.

In some instances, large organizations that manage transactions between many legal entities may use blockchain-based technology to keep track of the transactions. While the legal entities may fall under the same corporate umbrella, each of the legal entities may maintain its own distributed ledger to keep track of any transactions it performs and its current financial status. Given the large amount of computational resources that are required to maintain just one distributed ledger, maintaining separate blockchains for many legal entities may incur a large processing burden on the organization.

The processing burden may require the corporation to utilize more processors and memory and, in some cases, may cause the corporation to maintain data warehouses or server farms that are dedicated to storing the transaction data. Such processing may also incur increased electricity usage given the amount of energy nodes expend as they verify block instances each time they add a new block to the respective blockchain. This problem may be exacerbated as the number of legal entities and corresponding blockchains that fall under the organization increases given the large number of nodes that are often needed to maintain the integrity of individual blockchains to stop malicious parties from gaining control of the data.

SUMMARY

For the aforementioned reasons, there is a need for a method to maintain a centralized blockchain that is accessible to multiple legal entities under a corporate umbrella without requiring each legal entity to maintain its own blockchain. The method aims to provide functionality that enables each legal entity to contribute to the blockchain while maintaining the privacy of any transactions to which

2

it commits. Disclosed herein are systems and methods for creating a hierarchy-based blockchain that enables related legal entities to append transaction blocks to a blockchain using a rule and/or hierarchy-based method. The systems and methods may also enable blocks to be added to the blockchain without a quorum of all of the nodes that maintain the blockchain verifying the authenticity of the transaction.

Implementations of the systems and methods described herein may integrate hierarchy files and scripted executable code (e.g., smart contracts) into a central blockchain so nodes that maintain the blockchain may more easily determine how to authenticate transactions between internal nodes. For example, if an investor node and an issuer node of the blockchain seek to enter into a transaction, the investor node may submit a transaction request to the other nodes that maintain the blockchain and the nodes may check the hierarchy and/or the rules of a smart contract that are stored on the blockchain to determine the requirements for the transaction to be approved (e.g., for a block instance to be appended to the blockchain). The nodes may determine that only a set of nodes at a predetermined tier of the hierarchy above the investor node need to approve of the transaction for the transaction to be authorized. The set of nodes may use smart contracts that are stored on the blockchain to check to make sure the terms of any transaction conform to established rules of the legal entities.

Because of the hierarchy and rule-based model, the systems and methods described herein may eliminate the need for individual legal entities to maintain their own blockchains or other ledger technologies and allow for a single blockchain to store transaction data for transactions between legal entities to be processed and approved. The rules that are stored on the blockchain may further enable the legal entities to ensure they can uniquely control the transactions into which they enter despite such transactions being managed by a global authority.

In one embodiment, a system comprises a plurality of nodes each corresponding to a computing device having a processor and memory to store data, wherein each processor is configured to: receive a transaction request from a first node within the plurality of nodes, the transaction request corresponding to a pending transaction between the first node and a second node within the plurality of nodes via a third node within the plurality of nodes, the transaction request comprising a set of transaction attributes, wherein the first node is not in direct communication with the second node; identify a blockchain associated with the pending transaction, the blockchain including a first block instance having a hierarchy file indicating a hierarchy among the first node, the second node, and the third node; in response to the hierarchy file indicating that transaction requests from the first node must be approved by the third node, identify a second block instance having an executable file to approve the transaction request received from the first node; execute the executable file, wherein the executable file is configured to retrieve data from a data source associated with the third node and analyze the retrieved data associated with at least one of the first node, the second node, third node, or the set of transaction attributes to approve or deny the transaction request; and in response to receiving an indication of approval from the executable file, append a third block instance comprising the set of transaction attributes to the blockchain.

In another embodiment, a method comprises: receiving, by a consensus of a plurality of nodes, a transaction request from a first node within the plurality of nodes, the transac-

3

tion request corresponding to a pending transaction between the first node and a second node within the plurality of nodes via a third node within the plurality of nodes, the transaction request comprising a set of transaction attributes, wherein the first node is not in direct communication with the second node; identifying, by the consensus of the plurality of nodes, a blockchain associated with the pending transaction, the blockchain including a first block instance having a hierarchy file indicating a hierarchy among the first node, the second node, and the third node; in response to the hierarchy file indicating that transaction requests from the first node must be approved by the third node, identifying, by the consensus of the plurality of nodes, a second block instance having an executable file to approve the transaction request received from the first node; executing, by the consensus of the plurality of nodes, the executable file, wherein the executable file is configured to retrieve a data from a data source associated with the third node and analyze data associated with at least one of the first node, the second node, third node, or the set of transaction attributes to approve or deny the transaction request; and in response to receiving an indication of approval from the executable file, appending, by the consensus of the plurality of nodes, a third block instance comprising the set of transaction attributes to the blockchain.

It is to be understood that both the foregoing general description and the following detailed description are exemplary and explanatory and are intended to provide further explanation of the invention as claimed.

BRIEF DESCRIPTION OF THE DRAWINGS

The accompanying drawings constitute a part of this specification, illustrate an embodiment of the invention, and together with the specification, explain the invention.

FIG. 1 illustrates implementation of a hierarchy-based blockchain system, according to an embodiment.

FIG. 2 illustrates appending block instances to a blockchain, according to an embodiment.

FIG. 3 illustrates node hierarchy of a hierarchy-based blockchain system, according to an embodiment.

FIG. 4 illustrates a flowchart depicting operational steps for implementing a hierarchy-based blockchain system, according to an embodiment.

FIG. 5 graphically illustrates implementing a hierarchy-based blockchain system, according to an embodiment.

DETAILED DESCRIPTION

Reference will now be made to the exemplary embodiments illustrated in the drawings, and specific language will be used here to describe the same. It will nevertheless be understood that no limitation of the scope of the invention is thereby intended. Alterations and further modifications of the inventive features illustrated here, and additional applications of the principles of the inventions as illustrated here, which would occur to a person skilled in the relevant art and having possession of this disclosure, are to be considered within the scope of the invention.

FIG. 1 illustrates components of a hierarchy-based blockchain system 100, according to an embodiment. The system 100 may comprise an analytics server 110a, a system database 110b, distributed peer nodes 120a-c (hereinafter referred to as peer nodes 120 or peer node 120), a client device 140, and a network 150. Aspects of the system 100 may be configured to employ and manage a system blockchain, sometimes referred to in the art as a “distributed

4

ledger,” and may include blockchain-based distributed ledger software (e.g., Hyperledger, Ethereum, Openchain, and TerraLedger). Network 150 may be any synchronous or asynchronous network. The system blockchain may operate as a distributed database that stores data records associated with users and transactions between peer nodes 120, where the data records stored on the system blockchain may be blocks of data (e.g., block instances or blocks) that are hosted (e.g., locally stored or otherwise in possession of each distributed, such as being remotely hosted or hosted in a cloud) on distributed peer nodes 120. The system database 110b may also store records associated with transactions between peer nodes 120. Data stored in records within system databases 110b may vary from the data stored in blocks of the system blockchain hosted on peer nodes 120. Furthermore, each block instance may not be accessible to other peer nodes 120, however all block instances may be accessed by the analytics server 110a and/or may be stored in the system database 110b. In some embodiments, the analytics server 110a may generate a duplicate of one or more block instances within the blockchain and store said block instances in the system database 110b.

While the analytics server 110a may dictate accessibility and transmit instructions to other parties within system 100, each peer node within the distributed peer nodes 120 (e.g., creators of the block instance) or the client (via the client device 140) may prevent others within the system 100 from accessing at least portions of the data within one or more block instances. For example, while generating a block instance, a peer node 120 may encrypt a portion of the block instance so one or more peer nodes 120 may be unable to access the encrypted portion. Peer nodes 120 of a transaction may encrypt transaction data of a block instance so entities that were not privy to the transaction may not be able to view or access the transaction details. Consequently, only the peer nodes 120 that were privy to the transaction may access the data (e.g., by using their respective PKI key).

The analytics server 110a may be any computing device comprising a processor and non-transitory machine-readable storage capable of executing the various tasks and processes described herein. Non-limiting examples of such computing devices may include workstation computers, laptop computers, server computers, laptop computers, and the like. In some embodiments, the analytics server 110a may be or include one or more peer nodes of peer nodes 120 that maintain the system blockchain. The analytics server 110a may generate and display a user interface on the client device 140. An example of a user interface generated and hosted by the analytics server 110a may be a website. The analytics server 110a may host a website accessible to end-users, where the content presented via the various webpages may be controlled based upon end-users’ accounts and/or account permissions.

While the system 100 includes a single analytics server 110a, in some configurations, the analytics server 110a may include any number of computing devices operating in a distributed computing environment.

The analytics server 110a may execute software applications configured to display the user interface (e.g., host a website), which may generate and serve various webpages to the client device 140 or each peer node of the distributed peer nodes 120. The website may be used to generate and access data stored on the system database 110b or a blockchain hosted by nodes 120. In some implementations, the analytics server 110a may be configured to require user authentication based upon a set of user authorization credentials (e.g., username, password, biometrics, crypto-

graphic certificate, and the like). Such authorization credentials may correspond to a peer node account or a user account that is otherwise associated with a peer node 120. In such implementations, the analytics server 110a may access the system database 110b, which may be configured to store user credentials. The analytics server 110a may be configured to reference the stored user credentials to determine whether a set of entered credentials (purportedly authenticating the user) match an appropriate stored set of credentials that identify and authenticate the user.

The system database 110b can be a dynamic database that includes transaction records that the system database 110b receives from the peer nodes 120 and from various other sources (e.g., data source providers). The system database 110b can be a graph database, MySQL, Oracle, Microsoft SQL, PostgreSQL, DB2, document store, search engine, key-value store, etc. The system database 110b may be configured to hold any amount of data and can be made up of any number of components. Transaction records stored in the system database 110b may comprise one or more data fields (e.g., attributes and/or attribute values) containing transaction-identifying hash values generated by the analytic server 110a according to a hashing algorithm implemented by the peer nodes 120. When a new transaction record containing a machine-readable computer file (e.g., PDF, DOC, XSL), is received, the analytic server 110a may store the new transaction record in the system database 110b. The analytic server 110a may also generate a hash value for the new transaction record and store the hash value in the system database 110b. The hash value may be a unique identifier for the particular transaction record and may be used by various computing devices of the system 100, such as the system database 110b, to reference the computer file or metadata describing the computer file. The computer file may be stored in the system database 110b and/or on a block instance of the system blockchain that is hosted on peer nodes 120.

In some embodiments, the system database 110b may include a hierarchy file indicating a hierarchy of the different nodes of the system 100. The hierarchy file may include information indicating which nodes are at which tier of the hierarchy and the rules associated with each tier (e.g., the number of nodes of each tier that must approve the transaction for the respective tier to authorize a transaction). The hierarchy file may also include indications of the transaction attributes that are relevant to each tier (e.g., the transaction attributes that the peer nodes 120 may provide to nodes of the tier to authorize a transaction). As described herein, a similar hierarchy file may be stored on the system blockchain.

The system database 110b may be hosted on any number of computing devices comprising a non-transitory machine-readable storage medium and may be capable of performing the various tasks described herein. The system database 110b may be accessed by the peer nodes 120 via the network 150.

Peer nodes 120 may represent one or more institutions under the same corporate umbrella (e.g., each peer node 120 may be a computing device or group of computing devices of an affiliate of the same company), entities that are associated with the corporate umbrella, and/or entities that are outside of the corporate umbrella. Peer nodes 120 may represent any group of computing devices (e.g., any group of computing devices that perform transactions with each other and maintain a blockchain for such transactions). Each peer node 120 may have its own status within the corporate umbrella or may be outside of the corporate umbrella

altogether (e.g., investor nodes may not be under the corporate umbrella of different custodian nodes or issuer nodes). For example, as is described in more detail with respect to FIG. 3, peer nodes 120 may be investor nodes, broker dealer nodes, prime dealer nodes, global custodians, sub-custodians, central security depository nodes, etc. A peer node 120 may be any computing device comprising a processor and a non-transitory machine-readable storage medium capable of performing the various tasks and processes described herein. Non-limiting examples of a peer node may be a workstation computer, laptop computer, tablet computer, and server computer. Although three peer nodes are shown, any number of peer nodes may be utilized. Although the peer nodes 120 are described as storing blocks of the blockchain in FIG. 1, other computing devices, such as the analytics server 110a, may host blocks of the blockchain or host a corresponding blockchain. Each peer node 120 may locally store an instance of the system blockchain in the storage medium of the system blockchain, and may further execute a software application that instructs the peer node 120 to generate and query blocks within the locally stored blockchain instance as described herein.

In some embodiments, the analytics server 110a may generate and host webpages for a client device 140. The client device 140 may be one or more computing devices that can access such webpages by signing into accounts that are associated with individual peer nodes 120. The client device 140 may be any type of computing device. The analytics server 110a may generate webpages based upon a peer node's 120 role within the system 100 (e.g., investor, global custodian, broker dealer, prime broker, global custodian, sub-custodian, CSD, or issuer). For example, the analytics server 110a may identify the account of a peer node 120 (e.g., an account being accessed by the client device) that has logged into the system to initiate a transaction or view the attributes of the transaction. The analytics server 110a may select a user interface to present to the node based on the account characteristics that indicate the type or role of the peer node 120. For instance, the analytics server 110a may generate a user interface that allows investor nodes to initiate transactions, a user interface that allows custodian nodes to maintain or approve transactions, a user interface that enables issuers to provide securities for sale or to view potential transactions initiated by investor nodes, etc. In such implementations, the characteristics identifying each node's role may be defined by data fields and input fields in account records stored in the system database 110b. The analytics server 110a may authenticate an account and identify the account's role by executing an access directory protocol (e.g. LDAP). The analytics server 110a may generate webpage content, access or generate data stored in the system database 110b, and access or generate data stored in the blockchain instances, according to the account's role defined by the account record in the system database 110b.

In operation, in some embodiments, a user at the client device 140 may provide an input into a user interface generated by the analytics server 110a to initiate a transaction between a first and second peer node 120. The transaction may be a security purchase or a bond purchase and have various attributes (e.g., transaction type (e.g., securities purchase, bond purchase, etc.), initiation date, amount, transacting peer, currency, etc.). Upon receiving the transaction by the client device 140 for the first peer node 120, the analytics server 110a may forward the transaction request to the other peer nodes 120. For example, the client device 140 may initiate, via a user interface, a transaction between an investor node and an issuer node by logging into

an account associated with the investor node and providing an input that causes the investor node to purchase an asset such as a bond or a number of shares that were issued by the issuer node. The client device **140** may input the transaction attributes of the transaction into the user interface and the analytics server **110a** may forward the transaction request to one or more prime broker nodes and/or one or more global custodian nodes of the peer nodes **120** (e.g., a third peer node **120**) for processing. The analytics server **110a** may forward the transaction request to any number of peer nodes **120** except for, in some cases, the second peer node **120** (e.g., the issuer node). In some cases, the analytics server **110a** may forward the transaction request to a consensus of the peer nodes **120**. The network may allow this transaction to be broadcast to all relevant nodes (e.g., nodes of the transaction and/or nodes that are verifying the transaction) simultaneously or sequentially, depending on network configuration.

The consensus of the peer nodes **120** may receive the transaction request and identify a blockchain that they maintain and that is associated with the first peer node **120**, the second peer node **120**, and the third peer node **120**. The consensus may identify a block instance from the blockchain that stores a hierarchy file. The hierarchy file may indicate different tiers and/or groups of peer nodes **120** including a hierarchy among the first peer node, the second peer node, and/or the third peer node. For example, the hierarchy file may indicate that the first peer node is an investor peer node **120** at the bottom of the hierarchy and the second peer node is an issuer peer node at the top of the hierarchy. The hierarchy file may indicate that the third peer node **120** is a global custodian peer node among multiple global custodian peer nodes **120** that are on an upper tier of the hierarchy.

The hierarchy file may indicate that the global custodian peer nodes **120** that are on the upper tier of the hierarchy must approve any transaction that is initiated by the first peer node **120**. Accordingly, the consensus of the plurality of nodes may identify a block instance that stores files or that is otherwise associated with the global custodian peer nodes **120** on the system blockchain. The block instance may be or include a smart contract that, upon execution, causes the consensus to retrieve data from a data source (e.g., a blockchain or a central database) associated with the global custodian peer nodes **120** that includes one or more rules indicating which transactions to allow and/or decline. The consensus of the peer nodes **120** may provide the global custodian nodes with transaction attributes that correspond to the custodian nodes. Each of the global custodian peer nodes **120** may compare the transaction attributes of the transaction to the rules and, responsive to one of the rules being satisfied, transmit indications to the plurality of nodes to indicate their approval. However, responsive to a custodian peer node **120** determining none of the rules are satisfied, the custodian peer node **120** may send back an indication that it does not approve of the transaction. The consensus of the peer nodes **120** may maintain a count for each custodian peer node **120** that transmits an indication authorizing the transaction.

The consensus of the peer nodes **120** may determine whether a quorum of the custodian peer nodes **120** has approved of the transaction by comparing the count to a user set threshold. Upon determining the count exceeds the threshold, the consensus of the peer nodes **120** may determine the custodian peer nodes **120** have approved of the transaction and transmit a signal to the second peer node **120** indicating the transaction was approved by the custodian peer nodes **120**.

The second peer node **120** may receive the indication and determine whether to approve of the transaction. The second peer node **120** may do so based on its own set rules or based on a user input. Upon the second peer node **120** approving the transaction, the second peer node **120** may transmit a signal to the consensus of the peer nodes **120** indicating the approval, and the consensus of the peer nodes **120** may append a block instance to the system blockchain indicating the transaction attributes and other data associated with the transaction (e.g., timestamp data, the peer nodes involved in the transaction, the votes of the global custodian peer nodes **120**, etc.). In some embodiments, the second peer node **120** and/or the consensus may transmit a signal to the analytic server **110a** indicating the approval and the transaction attributes of the transaction. The analytic server **110a** may store transaction data in the system database **110b** as a record to maintain a record of the transaction off-blockchain for added security such as to have data to compare against the system blockchain to check if the system blockchain was compromised.

When appending block instances to the system blockchain, the peer nodes **120** may generate block addresses for data to be retrieved from the respective blockchain. To do so, the peer nodes **120** may generate a hash value for the transaction record of the block instance, where the hash value or other identifier values may be a block address to reference the block instance within the blockchain. The peer nodes **120a-c** may generate the hash value for the transaction record by generating a hash based on the transaction record (e.g., based on all or a portion of the data of the transaction record) and the data of the immediately preceding block data or block address. This block address may then be stored into the system database **110b** in a document record along with the transaction record and any number of additional data field entries related to the transaction record.

In operation, the analytics server **110a** or peer nodes **120** may reference the block of the blockchain containing the file according to the block address. The analytics server **110a** or peer nodes **120** may generate additional blocks and corresponding block addresses on the system blockchain in a similar manner (e.g., generating a hash value for a block containing user data and then generating a new block address using the block address of the preceding block). Block addresses may be generated in any number of combinations of hashed block data and/or hashed block addresses from the new block and one or more preceding blocks, such that the address of the new block is dependent upon, or otherwise linked to, at least the immediately preceding block.

In some embodiments, a peer node **120** may generate new blocks on a locally stored instance of the system blockchain according to data received from the analytics server **110a** or other peer nodes **120**. The analytic server **110a** may query the block instances of the system blockchain according to a block address stored in the system database **110b**. When the analytics server **110a** executes the query of the blocks on the system blockchain, the analytics server **110a** may poll the peer nodes **120** to determine the most recent data on the system blockchain (e.g., latest valid blockchain).

In some embodiments, the analytics server **110a** (or peer nodes **120**) may ensure that the data of the system blockchain is accurate according to a voting mechanism encoded within the blockchain software executed by the peer nodes **120**. Each peer node **120** may receive a query for the block instance and block address, and return a response to the analytics server **110a** indicating whether the block address contains the same data. The analytics server **110a** may select

this method to combat possible fraud and to be certain that data in the blockchain is resistant to corruption, as each block instance on a quorum or consensus of the peer nodes **120** would need to be corrupted in the same way for a possible security breach. In this way, the analytics server **110a** may also be prevented from acting on obsolete data. For instance, a peer node **120** may attempt to modify information about a particular transaction. By modifying the information within the block instance, the hash value of said block instance may change, which would result in the block instance being disconnected from other block instances within the blockchain. Furthermore, when queried, other peer nodes **120** may indicate that the modified block instance does not match a version of the data stored on their respective nodes. As a result, the analytics server **110a** may determine that the modified block instance has been indeed tampered with. The peer nodes **120** may then refuse to use the modified block instance.

As described above, the process performed by the peer nodes **120** may be performed by a consensus of the peer nodes **120**. A consensus may be a majority, all, or a number above a predetermined threshold (which may be set and be adjusted by an administrator). The peer nodes **120** may each perform the process to verify the validity of a transaction between peer nodes **120**. In some embodiments, different nodes of the consensus may perform different steps of the process. Responsive to a consensus of the peer nodes **120** receiving an indication of approval of a transaction request, the consensus may append a block instance for the respective transaction to the blockchain maintained by the peer nodes **120**. In some cases, peer nodes **120** may in turn append block instances to blockchains maintained internally by the respective peer nodes **120**. Because the peer nodes **120** may append a block instance to the blockchain using the hierarchy-based method, the peer nodes may append the block instance to the blockchain without requiring a quorum, substantially lowering the required resources to maintain the blockchain.

Referring now to FIG. 2, a non-limiting example of appending block instances to a blockchain comprising different block instances is illustrated. As depicted in FIG. 2, a blockchain **208** comprising block instances **202a-202n** (collectively **202**) may include data **204a-204n** (collectively **204**) that enables information, such as transaction data, hierarchical data (e.g., a hierarchy file indicating a node hierarchy of the blockchain), smart contract data, machine-readable code/documents, and other metadata associated with one or more transactions of the peer nodes described above. The block instances **202** may also contain hash values **206a-206n** (collectively **206**) that are used to link each of the block instances to the preceding block instance, as understood in the art.

Peer nodes may generate (or instruct a blockchain service to generate) the block instance **202a** (e.g., the genesis block). The peer nodes may receive data **204a** from a first peer node or a first computing device via a GUI provided by an analytic server on the first computing device or peer node. For example, an administrator using the first computing device may log in to a website hosted or otherwise associated/managed by the analytic server and transmit data **204b** (e.g., a transaction record) to other peer nodes. The peer nodes may verify the data **204b** against data of a corresponding transaction record. Responsive to determining the data matches, the peer nodes may generate a block instance for the blockchain that they maintain. Upon generation of the block instance **202b**, the peer nodes may generate the hash value **206b** based on the data **204b** (and/or data of the

immediately previous block instance), an identifier of the first computing device, and/or other identifier information (e.g., time stamp and/or geolocation information). The identification information may be used as a veracity scale factor that the information is true and accurate.

The peer nodes may generate (or instruct a blockchain service to generate) the block instance **202c**. The peer nodes may receive data **204c** from a second computing device (e.g., a second peer node) via a GUI provided by the analytic server on the second computing device. For example, an administrator using the second computing device may log in to a website hosted or otherwise managed by the analytic server and the second computing device may transmit data **204c** to the peer nodes. The peer nodes may generate a hash value **206c** based on the data **204c** and other information as described above.

The hash value **206c** may be based on the hash value **206b** and/or the data **204c**. The peer nodes may incorporate the hash value **206b** into the hash value **206c** to append the block instance **202c** to the block instance **202b**. The peer nodes may subsequently poll all the peer nodes to ensure the highest integrity of the blockchain by appending the latest block instance to the latest valid blockchain instances (e.g., the last blockchain for which there was a quorum). Using this method, block instances within the blockchain **208** may be appended to the preceding block instance. The peer nodes may generate block instances **202c-n** using the same methods explained above to continuously update the blockchain **208**. As depicted, block instances **202a**, **202b**, **202c**, and **202n** are connected because of synchronization of hash values **206a**, **206b**, **206c**, and **206n**.

Modification of data within a block instance may disconnect that block instance from the blockchain. The peer nodes may use this method to combat possible fraud or unauthorized modification of the data within blockchain instances. For example, if the second administrator using the second computing device modifies data **204b** within block instance **202b**, the hash value **206b** will also be modified. As explained above the hash value **206b** may be based on (or at least partially based on) data **204b**; therefore if data **204b** is modified, hash value **206b** will also be modified. Modification of the data **204b** or the hash value **206c** may break the link between block instance **202b** and block instance **202c** because hash value **206c** is at least in part generated based on hash value **206b**.

FIG. 3 illustrates a non-limiting example of a node hierarchy (e.g., node hierarchy **300**). The node hierarchy **300** may be an example hierarchy showing communicative relationships between a plurality of nodes of a hierarchy-based blockchain system. Node hierarchy **300** may include investors **302a-c** (hereinafter referred to as investors **302** or investor **302**), broker dealer **304**, prime dealer **306**, global custodian **308**, sub-custodian **310**, central securities depository (CSD) **312**, and issuer **314**, in some embodiments. Node hierarchies may include any number of nodes or different types of entities. An investor **302** may be or include a node that initiates a transaction by purchasing an asset such as a bond or stock. A broker dealer **304** may be or include a node that executes transactions for assets. In some instances, a broker dealer **304** may hold or store assets themselves. A prime dealer **306** may be or include a node that executes transactions but provides custody for investors **302**.

A global custodian **308** may be or include a node that provides portfolio custody services for clients in multiple markets or across different blockchain environments. A sub-custodian **310** may be or include a node that is the legal owner of an asset. A sub-custodian may hold or store assets

11

on behalf of clients such as investors **302**. CSD **312** may be or include a node that stores and/or maintains a register of the owners of assets on behalf of one or more issuers **314**. Issuer **314** may be or include a node from which investors **302** purchase assets. Each entity of node hierarchy **300** may be or include a separate node that may communicate with other nodes within node hierarchy **300**, as illustrated. Node hierarchies may be organized in any fashion.

Various parties shown in the node hierarchy **300** may belong to different entities and may utilize different computing devices to interact with the same blockchain. As discussed herein, these parties may have competing and/or conflicting interests. Therefore, data generated by different parties may not be treated equally. A single blockchain may be configured using the methods described herein, such that the investor **302a** and the sub-custodian **310** can add data to the blockchain using different blockchain consensus. That is, the blockchain is configured to use different protocols for adding data based on a node's position within the node hierarchy **300**. The node hierarchy **300** may be revised by a system administrator.

FIG. 4 illustrates execution of a method **400** for implementing a hierarchy-based blockchain, according to an embodiment. Other embodiments may comprise additional or alternative steps, or may omit some steps altogether. Each of the steps of the method **400** may be performed (in some cases separately) by one or a combination of a server and/or any of a plurality of nodes that maintain a blockchain.

At step **402**, a consensus of a plurality of nodes may receive, from a first node, a transaction request to initiate a pending transaction between the first node and a second node of the plurality of nodes. The transaction request may include transaction attributes (e.g., amount, time, parties to the transaction, etc.) for a potential transaction between the first node and the second node such as a bond purchase or a purchase of shares. As explained above, a server associated with the plurality of nodes may generate a graphical user interface to provide transaction record services to the first node. In an embodiment, the graphical user interface may be a website hosted by a server or one of the peer nodes, which may be available to each of the plurality of nodes. The purpose of said website may be to collect transaction data such as data in transaction records, and to provide a platform to securely upload transaction data, and/or to possibly retrieve transaction data from the peer nodes.

The graphical user interface may act as an intermediary between different nodes involved in a transaction and may be a central and "one-stop" website for facilitating transactions between peer nodes, storing data of the transactions, and/or retrieving such data. While the described graphical user interface is described herein as a central management tool, neither the peer nodes nor the graphical user interface deviate from the spirit of the distributed nature of the blockchain technology. The peer nodes may provide the management tool to reduce the computational burden of keeping record within different and unrelated databases. Blockchain and block instances may be stored in individual peer node databases.

The different nodes of the plurality of nodes may have different types of accounts for the website. Types of accounts may be, for example, investor, broker dealer, prime broker, global custodian, sub-custodian, central security depositories (e.g., securities databases), and/or asset issuer. As described above, each type of account may have different responsibilities for managing transactions that are executed and maintained on the blockchain and/or be responsible for verifying different transaction attributes. In some instances,

12

the different types of accounts may be dedicated to different types of assets or perform different functions when facilitating transactions between nodes that maintain the blockchain.

A third node of the plurality of nodes (e.g., a global custodian or a prime broker) may be dedicated to verifying or enabling transactions between nodes of the plurality of nodes. The third node may receive the transaction request from the first node, in some cases via a private communications channel the third node shares with the first node. The third node may receive and forward the request to other nodes of the plurality of nodes to determine whether to authorize the transaction. In some embodiments, the third node may send a prompt to the second node to indicate the first node is initiating a transaction and the terms of the transaction. Consequently, the first node may initiate a transaction with the second node without any direct communication with the first node, enabling the third node to facilitate whether to allow the transaction to be authorized.

For example, an investor node may submit a transaction request to initiate a bond purchase from a custodian node. The custodian node may forward the request to other nodes that manage the blockchain and send a prompt indicating the transaction request and its transaction attributes to the issuer node that issued the bond.

Upon receiving the transaction request, at step **404**, the consensus of the plurality of nodes may identify a blockchain that is associated with the pending request. The blockchain may be a blockchain that is maintained by the plurality of nodes including, for example, the first node, the second node, and/or the third node. The blockchain may include transaction data of transactions that are processed between the nodes that maintain the blockchain and other data associated with the nodes (e.g., smart contracts, geographical location information, rules to allow for transactions to be authorized, financial data, etc.).

The blockchain may include a block instance that contains a hierarchy file. The hierarchy file may include a hierarchy of the nodes of the plurality of nodes with tiers that indicate permissions that are required for different nodes to append block instances to the blockchain. The data structure of the hierarchy file may be an organized list of the different nodes and their tiers, tags associated with the nodes indicating their tiers, various rules identifying the different devices within the tiers, etc. The nodes may be represented in the hierarchy by various identifiers (e.g., device identifiers, account identifiers, group entity identifiers, or any other type of identifier). For example, in one embodiment, the hierarchy file may include two tiers. A node in the bottom tier may require permission from a quorum or a predetermined number of nodes of the top tier for a transaction to be authorized. For instance, an investor node may be in the bottom tier of the hierarchy and prime broker nodes may be in the top tier. When the investor node initiates a transaction with an issuer node (which may also be in the top tier), the investor node may need permissions from a quorum of the prime broker and/or custodian nodes for the transaction to be allowed and for the consensus to append a block instance for the transaction to the blockchain.

The hierarchy may include more than two tiers of peer nodes. For example, a bottom tier may include investor nodes, the next tier may include global custodian nodes, the next tier may include sub-custodian nodes, the next tier may include CSD nodes, and the top tier may include issuer nodes. The hierarchy file may include different rules for the tiers such as a majority of the global custodian nodes must approve the transaction while only one of the sub-custodian

nodes must approve the transaction for the transaction to be allowed. The nodes of each tier may execute smart contracts on the system blockchain or otherwise analyze data retrieved from execution of the smart contract to determine whether the terms of the transaction comply with their associated rules and therefore determine whether to approve of the transaction. The hierarchy file may include any number of rules or tiers.

The hierarchy file may include rules that indicate which tiers of the hierarchy for the consensus of the plurality of nodes to identify. The rules may be associated with different transaction attributes that are associated with the individual tiers. For example, the hierarchy file may include rules where transactions that are initiated at specific times may be verified by a specific tier of the hierarchy or transactions that have specific transaction attributes (e.g., transaction type, amount, etc.) may be verified by a specific tier of the hierarchy. In some embodiments, nodes of individual tiers of the hierarchy may only add specific types of data to the blockchain. The hierarchy file may include any number or type of such rules.

The consensus of the plurality of nodes may identify the tiers of the nodes that are involved in the potential transaction (e.g., the first node, the second node, and/or the third node). The consensus may identify the identifiers for the first node, second node, and/or the third node and determine the tier of each node. Based on the third node having a higher tier than the first and/or second node, the consensus may determine that the third node or a quorum of nodes on the same tier as the third node needs to approve the transaction request for the transaction to be authorized.

In response to the hierarchy file indicating that transaction requests from the first node must be approved by the third node, at step 406, the consensus may identify a second block instance that has an executable file that is configured to approve transaction requests received from the first node. The consensus may identify the second block instance based on an identifier contained within the second block instance that indicates the block instance comprises data associated with the third node. The executable file may be a smart contract that is associated with the third node. The smart contract may be configured to retrieve data from a data source (e.g., a database or a blockchain) of the third node. The consensus may execute the executable file by inputting the set of transaction attributes into the executable along with identifiers of the first and/or second nodes.

At step 408, upon execution, the executable file may cause the third node or the consensus of nodes to retrieve data (e.g., a document, a machine-readable file, etc.) from the data source associated with the third node. The retrieved data may include financial data about the first, second, or third nodes, sets of rules that correspond to the nodes, workflow management data, etc. The executable file may retrieve such data and evaluate the transaction attributes included in the transaction request against the data. The executable file may determine whether the transaction attributes satisfy the rules to determine whether to approve the transaction request.

For example, the data may include a series of rules. The series of rules may correspond to the first node. Examples of such rules may include automatically accepting any transaction requests from the first node, a requirement that a user at the third node review and manually approve transactions from the first node, only accepting transactions from the first node that have attributes that fall within specific ranges (e.g., transactions that have values between 1000 and 4000), only accepting transactions for which the first node has enough

money on-hand to make the purchase, etc. Similarly, the data may include a series of rules that correspond to the second and/or third node. For example, the third node and/or the second node may be associated with rules that only allow transactions with a value less than a threshold to be authorized. The second node may be associated with rules that only allow a predefined number of shares to be purchased at once. The third node and/or the second node may be associated with any number of rules. The executable file may retrieve such rules and/or data about the first node and compare the two to determine whether to approve the transaction.

Each tier of nodes may be associated with different attributes of a transaction and rules associated with the particular attributes. For example, the sub-custodians may be associated with different transaction attributes than prime brokers and have rules that are associated with the specific attributes. Such attributes may be designated in a file stored on the system blockchain or in a system database such as in a hierarchy file. Upon initiating a transaction, the consensus of the plurality of nodes may identify transaction attributes of the transaction that correspond to each of the tiers of the hierarchy. The consensus may only provide the corresponding transaction attributes to the nodes of the respective tiers. In turn, the nodes may compare the corresponding transaction attributes to a set of rules (e.g., input the transaction attributes into a smart contract of the blockchain) to determine if the transaction attributes satisfy their respective rules. Responsive to each tier of nodes determining the transaction attributes satisfies its respective set of rules, the consensus may generate a signal indicating the transaction has been authorized. However, in some embodiments, responsive to a portion of the attributes not satisfying rules of any one of the tiers, the consensus may determine the transaction was not authorized and generate a rejection signal to send back to the peer node that initiated the transaction. Advantageously, by using the hierarchical account structure described herein to authorize transactions, each tier may only view the portions of the transaction that are relevant to the respective tier without gleaning a full view of the transaction, enabling the transaction to remain private between the transacting parties while ensuring the transaction complies with the rules of each party that facilitates the transaction.

The executable file may be configured to allow the transaction to be authorized responsive to determining a consensus of a set of peer nodes (e.g., a set of nodes within the plurality of nodes) has approved the transaction. The set of peer nodes may be identified in the rules or may be specified in the executable file. The consensus of the plurality of nodes may execute the executable file and poll the set of peer nodes asking whether the peer nodes approve of the transaction. The peer nodes may analyze the data associated with the transaction (e.g., any transaction attributes that correspond to the respective tier of the peer nodes) and, in some cases, any rules or data that the executable file has retrieved from the data source. In some embodiments, the peer nodes may retrieve such rules and/or data from the data source. Responsive to determining the transaction data satisfies the retrieved data and/or rules from the data source, the peer nodes may send indications of their approval to the nodes executing the executable file. However, responsive to determining the transaction data does not satisfy the retrieved data and/or rules, the peer nodes may send indications of their disapproval.

The peer nodes may be identified based on their place in the hierarchy. For example, the hierarchy of the hierarchy

file may have multiple tiers indicating different nodes' placement within the hierarchy. The executable may identify the set of nodes based on an indication to query the set of nodes that is within the top tier of the hierarchy. The executable may identify the indication and identify the nodes that are associated with the top tier of the hierarchy. In turn, the executable may cause the consensus of the plurality of nodes executing the executable to poll the nodes that are associated with the top tier of the hierarchy asking if they approve of the transaction.

The executable file may determine whether a quorum of the polled set of nodes has provided their approval. The quorum may be a predetermined number or percentage of the polled nodes. The executable file may maintain a counter indicating the number of nodes that have responded with their approval of the transactions, incrementing the counter for each positive response. The executable file may compare the counter to the predetermined number or percentage. Responsive to determining the count of the counter exceeds the predetermined number or percentage, the executable file may determine the transaction has been approved by the set of nodes. The executable file may cause the consensus to prompt the third node to indicate the approval.

Responsive to determining the rules have been satisfied and/or a quorum of the set of nodes has transmitted an approval message to the consensus of the plurality of nodes, the executable file may cause the consensus to prompt the second node asking whether the second node approves or denies the transaction request. The executable may cause the consensus to send a prompt indicating the transaction attributes of the transaction, that the executable file has approved of the transaction, and/or the rules that were satisfied for the approval. The executable may generate the approval request and transmit the generated request to the second node.

The second node may receive the prompt from the executable or the consensus and determine whether to approve of the transaction request. The second node may do so according to its own set of rules (e.g., rules indicating whether the second node can sell to the first node, whether the number of shares falls within an allowed range, whether the purchase price is too high, etc.) or automatically upon receiving the prompt. The second node may determine whether to approve of the transaction request and transmit a signal to the third node indicating its approval or disapproval.

Responsive to receiving the approval from the second node, the executable file may generate an indication of approval for the transaction. The indication of approval may include an indication of identifiers of the first node, second node, and/or third node, indications of the set of transaction attributes, and/or any other characteristics about the transaction. The executable file may transmit or cause a signal to be transmitted to the consensus of the plurality of nodes indicating the generated indication of approval.

At step 410, in response to receiving an indication of approval from the executable file, the consensus may append a third block instance comprising the set of transaction attributes to the blockchain. The approval may be an indication that the requirements of the executable file were satisfied, that a quorum of the peer nodes above the first and/or second nodes approved the transaction, the second node approved the transaction, or any other indication that the transaction was approved. The consensus may receive the approval and append a blockchain to the blockchain that is maintained by the plurality of nodes. Advantageously, the consensus may do so without obtaining approval from a quorum of all of the plurality of nodes. Based on the data of the hierarchy file and/or the executable file, the consensus

may append the block instance for the transaction to the blockchain-based only on the approval of one or a few of the nodes of the plurality of nodes. Thus, the systems and methods described herein allow for fewer computational resources to be used when approving transactions and appending blocks to a blockchain.

FIG. 5 visually depicts a non-limiting example of how the methods and systems described herein can be implemented. The sequence 500 may include a plurality of nodes 502 that facilitate a transaction or a purchase of a financial instrument between a first node 504 and a second node 506 of the plurality of nodes 502 without any direct communication between the first node 504 and the second node 506. The first node 504 may transmit a message to a set of nodes 508 (e.g., a consensus of the plurality of nodes 502) of the plurality of nodes 502 indicating a potential transaction with the second node 506. The transaction request may include a set of transaction attributes for the potential transaction such as the purchase price and/or the number of shares that are being purchased.

The set of nodes 508 may identify a blockchain 509 that the set of nodes 508 maintains with the first node 504 and the second node 506. The blockchain 510 may initially comprise a plurality of block instances 512a-c. The block instance 512a may include a hierarchy file 514 and a hash 516 of the hierarchy file 514 and any other data being stored in the block instance 512a. The block instance 512b may include a smart contract 518 and a hash 520 of the smart contract 518, other data of the block instance 512b, and/or the hash 516. The block instance 512c may include transaction data 522 for a transaction between nodes of the plurality of nodes 502 and a hash 524 of the transaction data 522, the hash 520, and any other data stored within the block instance 512c.

The set of nodes 508 may receive the transaction request from the first node 504 and identify the hierarchy file 514 from the blockchain 510. The hierarchy file 514 may include a hierarchy 526 of the nodes of the plurality of nodes 502 indicating the permissions each node has and/or the rules for appending block instances to the blockchain 510. The hierarchy 526 may include a first tier 528 and a second tier 530. The first tier 528 may include nodes 532a-c for which a quorum must approve of transactions that are initiated by nodes 534a-b of the second tier 530. Nodes 532a-c and/or nodes 534a-b may be nodes of the plurality of nodes 502. For example, according to the hierarchy 526, if the node 534a attempts to purchase a financial instrument from another node, a quorum of the nodes 532a-c must approve of the transaction for the transaction to be approved. The first node 504 and/or the second node 506 may be associated with or be in the first tier 528 or the second tier 530.

In some embodiments, for the quorum of the nodes 532a-c to approve of the transaction, the set of nodes 508 or the nodes 532a-c may execute the smart contract 518. By executing the smart contract, the set of nodes 508 or the nodes 532a-c may retrieve data from a data source 536. The data source 536 may be maintained or stored by the set of nodes 508 or one or more of the nodes 532a-c. The data may include a set of rules indicating which transactions to allow (e.g., which transaction attributes are allowable or which nodes can initiate transactions), account information about the first node 504, account information about the second node 506, etc. The nodes 532a-c may compare the transaction attributes of the potential transaction between the first node 504 and the second node 506 and the retrieved data.

Upon a quorum of the nodes 532a-c determining the transaction should be allowed (e.g., based on the set of rules), the set of nodes 508 may prompt the first node 504

and/or the second node 506 to indicate the transaction has been allowed and append a block instance 536 to the blockchain 510. The block instance 538 may contain transaction data 540 and/or a hash 542 of the transaction data 540, other data of the block instance 538, and the hash 542 of the previous block instance 512c. Thus, the hierarchical structure of the nodes that maintain the blockchain 510 may enable nodes to append block instances to the blockchain 510 based on the hierarchy instead of requiring a quorum of all nodes that maintain the blockchain 510, enabling block instances to be appended using different sets of rules and permissions instead of just based on an agreement between the plurality of nodes 502.

While the systems and methods described herein have been integrated into a blockchain system, the hierarchy model may also be implemented without a blockchain. For example, a processor may store a hierarchy file within memory that indicates the different tiers of different nodes of the hierarchy. Upon receiving a transaction request, the processor can distribute the transaction attributes of the transaction request to the nodes according to the hierarchy and the nodes may determine whether the respective attributes are acceptable or otherwise comply with a set of rules established by the set of nodes. Responsive to each of the nodes determining the transaction attributes satisfy its respective rules, the processor may authorize the transaction. However, responsive to one of the nodes or one of the tiers of nodes determining the respective attributes do not satisfy its respective rules, the processor may not authorize the transaction. Thus the systems and methods provide for a new method of authorizing transactions between entities using a hierarchical data structure.

In an example, a plurality of nodes may each maintain a database that stores files including various rules that correspond to different transaction attributes. The different nodes may each be associated with specific transaction attributes. A central server may maintain a file that includes a hierarchy indicating the transaction attributes with which the different nodes are associated. Each tier of the hierarchy may be associated with a different set of transaction attributes. When a node initiates a transaction, the server may send the transaction attributes for the transaction to the corresponding nodes. The nodes may compare the received transaction attributes to their stored rules. Responsive to each of the nodes approving or determining the received transaction attributes comply with the node's respective rules, the server may allow for the transaction to go through. However, responsive to one of the nodes determining the transaction attributes do not satisfy the respective node's rules, the node may transmit a signal to the central server and the central server may decline the transaction. In yet another example, for a transaction request, the central server may receive rules from the different nodes and apply the rules to the transaction attributes of the transaction request that correspond to each respective node. Responsive to determining the applied rules are satisfied, the server may enable the transaction. However, responsive to determining at least one of the rules is not satisfied, the central server may stop the transaction.

The foregoing method descriptions and the process flow diagrams are provided merely as illustrative examples and are not intended to require or imply that the steps of the various embodiments must be performed in the order presented. The steps in the foregoing embodiments may be performed in any order. Words such as "then," "next," etc. are not intended to limit the order of the steps; these words are simply used to guide the reader through the description of the methods. Although process flow diagrams may

describe the operations as a sequential process, many of the operations can be performed in parallel or concurrently. In addition, the order of the operations may be re-arranged. A process may correspond to a method, a function, a procedure, a subroutine, a subprogram, and the like. When a process corresponds to a function, the process termination may correspond to a return of the function to a calling function or a main function.

The various illustrative logical blocks, modules, circuits, and algorithm steps described in connection with the embodiments disclosed herein may be implemented as electronic hardware, computer software, or combinations of both. To clearly illustrate this interchangeability of hardware and software, various illustrative components, blocks, modules, circuits, and steps have been described above generally in terms of their functionality. Whether such functionality is implemented as hardware or software depends upon the particular application and design constraints imposed on the overall system. Skilled artisans may implement the described functionality in varying ways for each particular application, but such implementation decisions should not be interpreted as causing a departure from the scope of the present invention.

Embodiments implemented in computer software may be implemented in software, firmware, middleware, microcode, hardware description languages, or any combination thereof. A code segment or machine-executable instructions may represent a procedure, a function, a subprogram, a program, a routine, a subroutine, a module, a software package, a class, or any combination of instructions, data structures, or program statements. A code segment may be coupled to another code segment or a hardware circuit by passing and/or receiving information, data, arguments, parameters, or memory contents. Information, arguments, parameters, data, etc. may be passed, forwarded, or transmitted via any suitable means including memory sharing, message passing, token passing, network transmission, etc.

The actual software code or specialized control hardware used to implement these systems and methods is not limiting of the invention. Thus, the operation and behavior of the systems and methods were described without reference to the specific software code being understood that software and control hardware can be designed to implement the systems and methods based on the description herein.

When implemented in software, the functions may be stored as one or more instructions or code on a non-transitory computer-readable or processor-readable storage medium. The steps of a method or algorithm disclosed herein may be embodied in a processor-executable software module, which may reside on a computer-readable or processor-readable storage medium. A non-transitory computer-readable or processor-readable media includes both computer storage media and tangible storage media that facilitate transfer of a computer program from one place to another. A non-transitory processor-readable storage media may be any available media that may be accessed by a computer. By way of example, and not limitation, such non-transitory processor-readable media may comprise RAM, ROM, EEPROM, CD-ROM or other optical disk storage, magnetic disk storage or other magnetic storage devices, or any other tangible storage medium that may be used to store desired program code in the form of instructions or data structures and that may be accessed by a computer or processor. Disk and disc, as used herein, include compact disc (CD), laser disc, optical disc, digital versatile disc (DVD), floppy disk, and Blu-ray disc where disks usually reproduce data magnetically, while discs reproduce data optically with lasers. Combinations of

19

the above should also be included within the scope of computer-readable media. Additionally, the operations of a method or algorithm may reside as one or any combination or set of codes and/or instructions on a non-transitory processor-readable medium and/or computer-readable medium, which may be incorporated into a computer program product.

The preceding description of the disclosed embodiments is provided to enable any person skilled in the art to make or use the present invention. Various modifications to these embodiments will be readily apparent to those skilled in the art, and the generic principles defined herein may be applied to other embodiments without departing from the spirit or scope of the invention. Thus, the present invention is not intended to be limited to the embodiments shown herein but is to be accorded the widest scope consistent with the following claims and the principles and novel features disclosed herein.

While various aspects and embodiments have been disclosed, other aspects and embodiments are contemplated. The various aspects and embodiments disclosed are for purposes of illustration and are not intended to be limiting, with the true scope and spirit being indicated by the following claims.

What is claimed is:

1. A system comprising:

a plurality of nodes each corresponding to a computing device having a processor and memory to store data, wherein one or more processors of the plurality of nodes are configured to:

receive a transaction request from a first node within the plurality of nodes, the transaction request corresponding to a pending transaction between the first node and a second node within the plurality of nodes;

identify a hierarchy file indicating a hierarchy comprising a plurality of tiers, each tier of the plurality of tiers corresponding to a different set of nodes of the plurality of nodes and a different ranking for approving transactions, the first node in a first tier having a lower ranking than a second tier of the hierarchy of the hierarchy file;

in response to the hierarchy file indicating that the first node is in the first tier having the lower ranking than the second tier of the hierarchy, determine whether at least a consensus of a set of nodes of the second tier of the hierarchy approves the transaction request; and

in response to an indication of approval by at least a consensus of the set of nodes of the plurality of nodes within the second tier of the hierarchy, append a record for the pending transaction to a distributed ledger indicating completion of the pending transaction.

2. The system of claim 1, wherein each processor is further configured to:

identify a first record of a distributed ledger associated with the pending transaction based on an identifier of the second tier of the hierarchy in the first record, the first record having an executable file to approve the transaction request received from the first node;

execute the executable file, wherein the executable file is configured to retrieve data from one or more data sources associated with the set of nodes of the second tier of the hierarchy and analyze data associated with the set of nodes to approve or deny the transaction request; and

20

generate the indication of approval using the executable file.

3. The system of claim 2, wherein each processor is configured to analyze the data associated with the set of nodes to approve or deny the transaction request by:

applying a set of rules associated with the second tier of the hierarchy to one or more attributes of the pending transaction; and

wherein each processor is configured to generate the indication of approval using the executable file responsive to determining the one or more attributes of the pending transaction satisfy the set of rules associated with the second tier of the hierarchy.

4. The system of claim 3, wherein each processor is configured to apply the set of rules associated with the second tier of the hierarchy to a subset of attributes of the pending transaction that correspond to the second tier of the hierarchy.

5. The system of claim 2, wherein the data source is a second distributed ledger.

6. The system of claim 1, wherein each processor is further configured to:

transmit the transaction request to a server in response to the indication of approval.

7. The system of claim 1, wherein each processor is further configured to:

identify a first record of a distributed ledger associated with the pending transaction based on an identifier of the second tier of the hierarchy in the first record, the first record having an executable file to approve the transaction request received from the first node; and execute the executable file to poll the set of nodes of the second tier of the hierarchy for approval for the pending transaction.

8. The system of claim 7, wherein each processor is further configured to:

execute the executable file to poll one or more nodes of the first tier of the hierarchy for approval for the pending transaction.

9. The system of claim 7, wherein the executable file is configured to prompt the second node to receive an approval or denial of the transaction request.

10. The system of claim 1, wherein each processor is further configured to:

transmit a message indicating an approval of the pending transaction to each node of the plurality of nodes.

11. A method comprising:

a plurality of nodes each corresponding to a computing device having a processor and memory to store data, wherein one or more processors of the plurality of nodes are configured to:

receiving, by one or more processors of a plurality of nodes, a transaction request from a first node within the plurality of nodes, the transaction request corresponding to a pending transaction between the first node and a second node within the plurality of nodes; identifying, by the one or more processors, a hierarchy file indicating a hierarchy comprising a plurality of tiers, each tier of the plurality of tiers corresponding to a different set of nodes of the plurality of nodes and a different ranking for approving transactions, the first node in a first tier having a lower ranking than a second tier of the hierarchy of the hierarchy file;

in response to the hierarchy file indicating that the first node is in the first tier having the lower ranking than the second tier of the hierarchy, determining, by the

21

one or more processors, whether at least a consensus of a set of nodes of the second tier of the hierarchy approves the transaction request; and
 in response to an indication of approval by at least a consensus of the set of nodes of the plurality of nodes within the second tier of the hierarchy, appending, by the one or more processors, a record for the pending transaction to a distributed ledger indicating completion of the pending transaction.

12. The method of claim 11, wherein each processor is further configured to:

identifying, by the one or more processors, a first record of a distributed ledger associated with the pending transaction based on an identifier of the second tier of the hierarchy in the first record, the first record having an executable file to approve the transaction request received from the first node;

executing, by the one or more processors, the executable file, wherein the executable file is configured to retrieve data from one or more data sources associated with the set of nodes of the second tier of the hierarchy and analyze data associated with the set of nodes to approve or deny the transaction request; and

generating, by the one or more processors, the indication of approval using the executable file.

13. The method of claim 12, wherein analyzing the data associated with the set of nodes to approve or deny the transaction request comprises:

applying, by the one or more processors, a set of rules associated with the second tier of the hierarchy to one or more attributes of the pending transaction; and
 wherein generating the indication of approval using the executable file comprises generating, by the one or more processors, the indication of approval responsive to determining the one or more attributes of the pending

22

transaction satisfy the set of rules associated with the second tier of the hierarchy.

14. The method of claim 13, wherein applying the set of rules associated with the second tier of the hierarchy comprises applying, the one or more processors, the set of rules associated with the second tier of the hierarchy to a subset of attributes of the pending transaction that correspond to the second tier of the hierarchy.

15. The method of claim 12, wherein the data source is a second distributed ledger.

16. The method of claim 11, further comprising:

transmitting, by the one or more processors, the transaction request to a server in response to the indication of approval.

17. The method of claim 11, further comprising:

identifying, by the one or more processors, a first record of a distributed ledger associated with the pending transaction based on an identifier of the second tier of the hierarchy in the first record, the first record having an executable file to approve the transaction request received from the first node; and

executing, by the one or more processors, the executable file to poll the set of nodes of the second tier of the hierarchy for approval for the pending transaction.

18. The method of claim 17, further comprising:

executing, by the one or more processors, the executable file to poll one or more nodes of the first tier of the hierarchy for approval for the pending transaction.

19. The method of claim 17, wherein the executable file is configured to prompt the second node to receive an approval or denial of the transaction request.

20. The method of claim 11, further comprising:

transmitting, by the one or more processors, a message indicating an approval of the pending transaction to each node of the plurality of nodes.

* * * * *